

px,color:333333;

```
class A1,A2,A3,A4 componentRepo;  
class B,C,D,E,F,G,I cptTool;  
class note1,repoStructure note;  
class CR subgraphTitle;  
class CPTT cptTitle;
```

Problem Statement

Currently, performance testing in GitLab primarily relies on the GitLab Performance Tool (GPT) running against self-managed instances following reference architectures.

While comprehensive, this approach has significant limitations:

1. Delayed Detection: Performance issues often emerge only after MR has been merged
2. Extended Setup Time: Environment and data preparation requires approximately 2 hours before testing
3. Resource Intensive: Full instance testing demands significant computational resources
4. Limited Isolation: Difficult in attributing performance issues to specific components
5. Delayed Feedback: Teams must review numerous MRs to identify performance regressions
6. Limited Component Visibility: Lack of granular insights into individual component performance
7. Test Ownership Misalignment: Tests primarily developed by Dev Ex rather than component teams
8. Operational Overhead: Requires dedicated performance on-call rotations

Component-level performance testing addresses these challenges through isolated testing, accelerated feedback loops, and targeted performance analysis.

Goal

Develop a self-service performance testing framework that component teams can integrate to gain insights into individual component performance and detect some performance issues early in the development lifecycle.

Responsibilities

Team and roles	Responsibilities	Location

Performance Enablement

Owner

</td>
<td>

Maintains the core framework
Regularly updates to testing tools and infrastructure
Applies security patches and dependency updates to the framework
Maintains comprehensive documentation
Provides training for new teams to onboard
Shares best practices and lessons learned
Acts on any feedback provided by the respective component teams
Updates the framework to provide proper performance insights to various component teams

</td>
<td>Component Performance Testing Tool repository</td>
</tr>
<tr>
<td>

Respective Component Teams

Owner

</td>
<td>

Ensures the component meets the pre-requisites for adding component level performance test
Ensures any new feature of the component meets the pre-requisites for adding component level performance test
Maintains their specific performance test scenarios
Constantly monitors the performance of their component
Grows the test suite by adding more performance related tests
Update tests as the component interfaces changes
Monitors the performance on each MR runs and updates the MR accordingly to gain the right performance
Adjusts thresholds in tests based on performance requirement
Updates the component setup as the configuration of the component changes

</td>
<td>Respective Component repository</td>
</tr>
</table>

Limitation of Component Performance Testing tool

Component performance testing cannot detect issues related to:

Integration Bottlenecks: Performance problems emerging from component interaction
Data volume scaling problems: Degradation occurring only with production-scale data
Network latency effects: End-to-end latency issues not apparent in isolated testing
Cascading failures: System-wide issues triggered by component interdependencies

etc.

Lets say, any issues that may arise due to huge data or production environment like setup, would not be caught by component performance testing.

The tool focuses on identifying:

Throughput bottlenecks: Component-specific request handling limitations

Caching effectiveness: Performance impact of component-specific caching strategies

Error handling overhead: Performance degradation caused by excessive or inefficient error handling

Configuration-related performance issues: Suboptimal component configuration settings

Serialization/deserialization overhead: Data transformation inefficiencies

etc.

GPT testing will continue on current schedules to maintain comprehensive real-world scenario coverage.

Approach

Implement component-level performance testing at the MR level to identify performance regressions before production deployment.

Components meeting the pre-requisites can integrate this tool in their CI Pipeline

Component-specific docker-setup.yml and k6-test.js files would be residing in the component repository, managed by the development teams. The tool will spin up the containerized component and execute Grafana K6 tests against it in a controlled setup.

Prerequisites to use the tool

Components should have the following capabilities before starting to use the tool

Containerization Support: Dockerized implementation deployable in isolation

API/ Interface capability: Exposed interfaces using one of the protocol supported by GrafanaLabs K6

Mocking/Isolated Testing Capability: Testable in isolation or with interface mocking

Metrics Collection: Clear point for collecting performance metrics (endpoints, methods, etc.)

Tool Implementation Challenges

The following points represent challenges that may be faced while implementing the tool

1. Environment Isolation: Managing component dependencies in isolated testing
2. Test and Setup Orchestration: Maintaining separation between tool orchestration and component-specific files
3. Test Data Variability: Ensuring consistent performance metrics across test data
4. Resource Management: Efficiently allocating resources for MR-level testing
5. Metrics Collection: Gathering and analyzing performance metrics from containerized components

6. Baseline Comparison: Establishing reliable performance baselines for comparison
7. Integration with CI/CD: Seamless integration with existing CI/CD pipelines
8. Reporting Limitations: Extending beyond standard K6 reporting capabilities
9. Tool Generalization: Balancing component-specific needs with framework standardization

Self Service Challenges

Adoption challenges include:

Components lacking pre-requisites capabilities
Team bandwidth constraints for test development
Additional maintenance of environment setup and test scripts for developers
Learning curve for K6 test development
Competing priorities and deadlines

Implementation Approach

Phase 1: PoC, Gathering Requirements and Identifying component

1. Identifying Component for PoC
 1. Identify component which satisfies pre-requisites to use the tool
 2. Gather performance requirements from the stakeholder groups
2. Proof Of Concept
 1. Develop basic K6 test implementation for MR-level testing

Phase 2: Core Infrastructure, Framework Setup & Pilot

1. Component Testing Framework
 - Create a reusable framework for component-level performance testing
 - Support containerized component deployment using Docker/Docker Compose
 - Implement secure credential management (git-crypt)
 - Configure test runners with appropriate resources
2. Testing Tools Integration
 - Integrate k6 for HTTP-based performance testing
 - Configure xk6 extensions for enhanced metrics collection
 - Implement telegraf for container metrics collection
 - Set up InfluxDB for metrics storage
3. CI/CD Integration
 - Implement MR pipeline integration
 - Configure performance report generation
 - Establish MR feedback mechanisms
4. Metrics & Visualization
 - Define key performance metrics for components
 - Develop Grafana dashboards
 - Implement trend analysis for long-term monitoring
5. Pilot Component Selection
 - Select initial component for pilot (e.g., AI Gateway)
 - Document component-specific requirements
 - Implement component-specific test scenarios

Phase 3: Pilot, Gather Feedback and Enhance tools

1. Test Scenario Development
 - Add more tests to ai-assist repository
2. Feedback Collection
 - Gather feedback from ai-assist team
 - Refine testing approach based on feedback
 - Document lessons learned
3. Tool Enhancement
 - Create CI templates for component performance testing
 - Compare test results with runs on main branch
 - Create main branch baselines for the component
 - Document baseline performance
 - Implement automatic baseline updates

Phase 4: Expansion & Refinement

1. Additional Component Onboarding
 - Identify next components for onboarding by getting in touch with Support and knowing which component lags in performance
 - Provide onboarding documentation
 - Understand setup of component and kickstart the team by adding 1 test for their component
2. Framework Enhancements
 - Implement multi-component testing capabilities
 - Enhance reporting and visualization
 - Optimize resource utilization
3. Documentation & Self-Service
 - Create comprehensive documentation
 - Implement self-service onboarding
 - Provide templates and examples

Technical Implementation Details

Tech Stack

GitLab CI: Used for triggering multi project pipeline

Google Cloud Platform: Used to run docker container of a component in a GCP instance as well as to run test scripts from a separate GCP instance

Docker: Running Dockerized container on GCP instances

Ruby: Messaging report to create leaner report

Bash Scripts: Run gcloud commands to create various GCP resources

Telegraf: Send metrics to InfluxDB

InfluxDB: Store metrics in buckets

Grafana: Create dashboard using metrics in InfluxDB.

Architectural flow

The flow mentioned in architecture diagram can be explained as follows

The component repository contains the component setup file in the form of performance-

tests/setup/docker-compose.yml and the performance-tests/k6-test/test-file.js

The .gitlab-ci.yml file should include the performance testing job as shown in the projects README.

When a MR is created or a commit has been pushed to an existing MR, it would run the following jobs

- dotenv-var job: stores the performance-test directory as artifacts and also stores some dotenv vars

- tests:performance job: This triggers the downstream multi-project pipeline while passing some env vars

The downstream multi-project pipeline gets triggered in Component performance testing tool project which does the following

- Download artifacts saved by dotenv-var upstream job and performs some validation on the docker-compose.yml and test-file.js gfiles

- Creates a GCP instance, downloads few dependencies and spins up the component using docker container

 - Sends container metrics to InfluxDB

- Creates a GCP instance, downloads few dependencies and spins up a test runner container which runs the tests against the component docker container running on the other GCP instance

 - Sends test run metrics to InfluxDB

 - Extracts the test results and creates a leaner test report

- Posts a comment on the MR with the test result or updates an existing comment with the updated result

Developer can see the performance test results as a comment in their MR

MR cycle time impact

The current performance testing job in ai-assist takes ~10 mins to finish. So this has increased the MR cycle time by 10mins. Bear in mind that each performance test runs for 1 min, so the more tests we add the more time it would take for the job to run.

However, following can be optimized to bring the timing down in case it is required.

- installing dependencies
- running sequential tests

Metrics and Dashboard

Telegraf is used to send following metrics to influxDB

- Docker container metrics (CPU, Mem etc)
- Docker Logs

xk6-output-influxdb is used to send test summary metrics of the k6 test container to influxDB.

Following metrics to be gathered as part of the tool setup

1. Test Execution Specific Metrics:

- Response time (min, max, p95, p99)

- Throughput (requests per second)

- Error rate

Custom component metrics

Test duration

Setup time

Teardown time

2. Resource Utilization Metrics from Component:

CPU usage

Memory consumption

Network I/O

Disk I/O

3. Pipeline Metrics:

Job ID

SHA Commit

Grafana will be used to create dashboards using the metrics stored in InfluxDB.

Reporting

k6 by default generates test summary metrics which is ingested by the tool and a much more leaner reporting is created that is posted in MR as a comment by a bot. The example of the report will be

text

```
+-----+-----+-----+-----+-----+-----+
| NAME           | RPS | RPS RESULT | TTFB AVG | TTFB P90 | REQ STATUS | RESULT |
+-----+-----+-----+-----+-----+-----+
| v2codecompleti | 2   | 1.97 (> 2/s) | 11.77    | 15.47 (< 25ms) | 100%      | Passed |
+-----+-----+-----+-----+-----+-----+
```

This pipeline will also run on main branch of the respective component repository, which will be considered as a baseline for performance results. Future iteration will involve comparing the test results generated in MRs and providing variance as a results.

Data Storage & Analysis

1. Time-Series Data:

Store performance metrics in InfluxDB

Tag data with relevant metadata (commit SHA, environment, test type)

Set appropriate retention policies

2. Baseline Management:

Store baselines in both InfluxDB and JSON files

Update baselines automatically based on statistical analysis

Support version-specific baselines

3. Report Generation:

Generate comprehensive JSON reports

Create summarized reports for MR comments

Provide links to detailed dashboards

Rollout Strategy

```
<table>
<tr>
<th>Financial Quarter</th>
<th>Task</th>
</tr>
<tr>
<td rowspan="8">
```

```
FY26::Q1
</td>
<td>
```

```
Identify component which satisfies pre-requisites to use the tool
</td>
</tr>
<tr>
<td>
```

Create a POC demonstrating the component performance testing in action for AI Gateway

<https://gitlab.com/gitlab-org/quality/quality-engineering/team-tasks/-/issues/3339>

```
</td>
</tr>
<tr>
<td>
```

Gather performance requirements from AI Framework group and AI Model validation group

<https://gitlab.com/gitlab-org/quality/quality-engineering/team-tasks/-/issues/3310>

```
</td>
</tr>
<tr>
<td>
```

Create a reusable basic framework for component-level performance testing

Support containerized component deployment using Docker/Docker Compose
Implement secure credential management (git-crypt)
Generate a lean report which is posted as a comment in MRs

```
</td>
</tr>
<tr>
<td>
```

Create a tests:performance job in ai assist which leverages the component performance testing framework and runs the job on each MR pipeline with a single tests for code completion.

```
</td>
```


</tr>
<tr>
<td>

Gather feedback from ai-assist team to improve the framework and optimize their tests:performance job.

</td>
</tr>
<tr>
<td>Create basic documentation for ai-assist team</td>
</tr>
<tr>
<td>Pilot this framework in ai-assist team</td>
</tr>
<tr>
<td rowspan="8">

FY26::Q2

</td>
<td>Add additional performance tests to ai-assist repository</td>
</tr>
<tr>
<td>Plan for onboarding second component(Gitaly)</td>
</tr>
<tr>
<td>Improve the stability of the performance testing</td>
</tr>
<tr>
<td>Continue gathering feedback from the team</td>
</tr>
<tr>
<td>Develop dashboards in Grafana for performance metrics visualization</td>
</tr>
<tr>
<td>Enhance the framework by improving its generalizability</td>
</tr>
<tr>
<td>Enhance the framework based on collected feedback</td>
</tr>
<tr>
<td>Investigate performance-related support issues to identify the next suitable component for framework adoption</td>
</tr>
<tr>
<td rowspan="5">

FY26::Q3

</td>
<td>Begin onboarding a second component(Gitaly)</td>
</tr>

	Enhance onboarding documentation
	Tracking adoption and effectiveness through Grafana dashboards

Collaborate with the development team to ensure the new component meets the pre-requisites for onboarding.

	Perform framework maintenance as needed

FY26::Q4

	Integrate the framework by creating jobs in the new component repository that leverage the framework for MR testing
	Enhance the framework to accommodate specific component needs while maintaining generalizability
	Update documentation to support self-service for the new component
	Continue tracking adoption and effectiveness through Grafana dashboards
	Perform ongoing framework maintenance
	Providing initial support to the newly onboarded component

Performance Feedback Management Workflow

mermaid

flowchart TD

```

A[Development Team] -->|Posts feedback in| B["gperformanceenablement" Slack channel]
B -->|Manual invocation of| C[Performance Enablement Bot]
C -->|Creates issue in| D["Component Performance Testing Tool" repository]
B -->|Ongoing discussion in| E[Slack thread]

```

E -->|Bot captures and| F[Updates issue with thread comments]
C -->|Labels issue and| G[Tags Performance Enablement team]
G -->|Issues included in| H[Next milestone]

```
classDef slackNode fill:4A154B,stroke:4A154B,color:white;  
classDef botNode fill:E01E5A,stroke:E01E5A,color:white;  
classDef gitlabNode fill:FC6D26,stroke:FC6D26,color:white;  
classDef teamNode fill:36C5F0,stroke:36C5F0,color:white;  
classDef milestoneNode fill:ECB22E,stroke:ECB22E,color:white;
```

```
class A teamNode;  
class B,E slackNode;  
class C,F botNode;  
class D,G gitlabNode;  
class H milestoneNode;
```

Success Metrics

The success of this implementation will be measured by:

1. Early Detection: Number of performance issues caught at MR level
2. Developer Adoption: Number of teams actively using component performance testing
3. Performance Trends: Improvement in key performance metrics over time
4. Test Execution Time: Reduction in time required for performance testing
5. Integration Effectiveness: Seamless integration with CI/CD pipelines
6. Decision Impact: Number of data-driven decisions made using performance insights

Maintenance & Support

1. Framework Maintenance:
 - Performance Enablement team maintains the core framework
 - Performance Enablement team regularly updates to testing tools and infrastructure
 - Performance Enablement team applies security patches and dependency updates
2. Component Test Maintenance:
 - Component teams maintain their specific test scenarios
 - Component teams update tests as component interfaces change
 - Component teams adjust thresholds based on performance requirements
3. Documentation & Training:
 - Performance Enablement team maintains comprehensive documentation
 - Performance Enablement team provides training for new teams
 - Performance Enablement team shares best practices and lessons learned

References

Component Performance Testing Repository
k6 Documentation
Docker Compose Documentation
InfluxDB Documentation
Telegraf Documentation

SECTION: engineering/infrastructure-platforms/developer-experience/design-documents/performance_datastore/_index.md

Summary

This blueprint proposes a Performance Results Data store to build off of the GitLab Performance Tool by centralizing performance metrics in order to enable data-informed decisions, dynamic baselines, and integrate performance awareness throughout the development lifecycle.

Goals

- Create a centralized repository for all performance test results
- Enable programmatic access to performance data for analysis and visualization
- Enable trend analysis between different test runs, environments, and GitLab versions
- Provide a foundation for automated performance regression detection
- Integrate with existing CI/CD pipelines for automatic data collection
- Support both high-level aggregated metrics and detailed raw performance data
- Enable dynamic baseline creation based on historical data

Non-Goals

- Replacing existing performance testing tools: We'll enhance and integrate with tools like GitLab Performance Tool rather than replacing them
- Duplicating our monitoring infrastructure: We'll leverage our current monitoring solutions rather than creating parallel systems
- Reinventing visualization: We'll continue to make use of current visualization tools instead of creating custom alternatives
- Competing with real-time monitoring: We'll complement our existing real-time monitoring capabilities rather than replacing them
- Expanding beyond performance focus: We'll maintain a dedicated focus on performance metrics to ensure depth and relevance of insights

This initiative is about amplification and evolution of our performance capabilities, not replacement. We're building upon the solid foundation created by previous work to unlock new possibilities without discarding the valuable systems already in place.

Proposal

We propose building a Performance Results Datastore to enable cross functional analysis and interpretation of the performance metrics. Key capabilities will include:

- Support multiple types of performance tests
- Store results with rich metadata (GitLab version, environment details, test parameters)
- Provide flexible querying capabilities for various analysis needs

- Scale to accommodate growing volumes of performance data
- Integrate seamlessly with existing CI/CD pipelines

Implementation Approach

We'll leverage existing infrastructure (InfluxDB and Grafana) to create a proof-of-concept that enables:

- Storage of performance data from multiple sources
- Visualization through Grafana dashboards
- Programmatic access from CI/CD pipelines

Infrastructure Details

The following resources have been provisioned for this implementation:

- InfluxDB Instance
 - URL: <https://influxdb.quality.gitlab.net/>
 - Bucket name: perf-test-metrics
- Grafana Instance
 - URL: <https://dashboards.quality.gitlab.net/>
 - Connected to the InfluxDB instance for visualization of performance metrics

Architecture Overview

mermaid

flowchart LR

```
%% Define nodes with meaningful IDs
subgraph SRC["Performance Data Sources"]
    direction TB
    RA["Reference Architecture Runs"]
    MR["MR Pipeline Tests"]
    UX["UX Performance (GBPT)"]
end
```

```
subgraph STORE["Performance Datastore"]
    direction TB
    SS["Server-side Metrics"]
    FE["Frontend/UX Metrics"]
end
```

```
subgraph VISUAL["Analysis & Visualization"]
    direction TB
    CI["CI/CD Pipeline Feedback"]
    GRAF["Grafana Dashboards"]
end
```

```
subgraph CONS["Consumers"]
    direction TB
```

```
DEV["Engineering Teams"]
PERF["Performance Stakeholders"]
end
```

```
%% Define the flow connections with better spacing
```

```
RA --> SS
```

```
MR --> SS
```

```
UX --> FE
```

```
SS --> CI
```

```
SS --> GRAF
```

```
FE --> CI
```

```
FE --> GRAF
```

```
CI --> DEV
```

```
GRAF --> DEV
```

```
GRAF --> PERF
```

```
%% Style the different components
```

```
classDef sources fill:e1f5fe,stroke:0288d1,stroke-width:2px,rx:10px,ry:10px
```

```
classDef store fill:fff8e1,stroke:ffa000,stroke-width:2px,rx:10px,ry:10px
```

```
classDef visual fill:f3e5f5,stroke:7b1fa2,stroke-width:2px,rx:10px,ry:10px
```

```
classDef consumers fill:ffebee,stroke:c62828,stroke-width:2px,rx:10px,ry:10px
```

```
classDef useCases fill:e0f7fa,stroke:00838f,stroke-width:1px
```

```
classDef ucItem fill:white,stroke:none
```

```
classDef ucLabel fill:none,stroke:none
```

```
classDef nodes fill:white,stroke-width:1px
```

```
class SRC sources
```

```
class STORE store
```

```
class VISUAL visual
```

```
class CONS consumers
```

```
class RA,MR,UX,SS,FE,CI,GRAF,DEV,PERF nodes
```

Key Use Cases

- MR Pipeline Performance Validation
- Trend Analysis
- Dynamic Baselines
- Regression Detection

Data Retention

Performance testing across numerous MRs will generate substantial data volumes. We need a thoughtful retention strategy to balance analytical value with storage constraints. Potential approaches include:

1. Selective Storage - Only persist results from merged MRs, treating pipeline runs on unmerged MRs as transient data

2. Data Aggregation - Implement a periodic process to consolidate historical data, preserving trends while reducing granularity of older measurements
3. Time-based Retention - Maintain full fidelity for recent data, for example 30-90 days, then progressively reduce resolution for older data
4. Significance-based Pruning - Retain all data points that represent significant changes or anomalies, while sampling or aggregating data that follows expected patterns
5. Environment-based Policies - Apply different retention rules based on the source environment, for example longer retention for Reference Architecture runs vs. MR pipeline tests.

Our initial implementation will focus on establishing the core infrastructure while we evaluate these approaches based on actual usage patterns.

Sample Workflows

As a developer, I want to know if my change affects performance

```
mermaid
sequenceDiagram
    participant DS as InfluxDB
    participant MR as MR Pipeline
    participant DEV as Developer

    note over MR: Run performance test
    MR->>DS: Query baseline results
    DS->>MR: Return historical data
    MR->>DS: Write results
    note over MR: Determine Pass/Fail status
    MR->>DEV: Report status
```

As a Stakeholder, I want to be able to investigate performance trends

```
mermaid
sequenceDiagram
    participant UX as UX Performance Run
    participant RA as Reference Architecture Run
    participant MR as MR Pipeline
    participant DS as InfluxDB
    participant GF as Grafana
    participant S as Stakeholder

    MR->>DS: Write results
    RA->>DS: Write results
    UX->>DS: Write results
    DS->>GF: Results
    GF->>S: Visualized Results
```

Alternative Solutions

1. Run a baseline run and the performance run every MR
 - Pros:
 - Provides immediate, direct comparison without relying on historical data
 - Guarantees latest reference point for comparison
 - Eliminates concerns about environmental or temporal variations
 - Cons:
 - Dramatically increases CI resource consumption and pipeline duration
 - Creates redundant test runs of the same baseline code
 - Doubles the testing time for every performance-relevant MR
 - Scales poorly as more performance tests are added to the suite
 - Use a hard coded baseline
2. Use Static, Hard-Coded Baselines
 - Pros:
 - Simple implementation with minimal infrastructure needs
 - Consistent reference points for comparison
 - Low maintenance overhead for implementation
 - Cons:
 - Quickly becomes outdated as the application evolves
 - Fails to account for legitimate performance changes over time
 - Requires manual updates to adjust expectations
3. Use Existing Per-Environment Prometheus Instances
 - Pros:
 - Leverages existing monitoring infrastructure
 - Data already collected and available
 - Cons:
 - Data is isolated within each environment
 - Added complexity on the CI run to determine which data source to use
 - Test run will vary, will the needed data be present?
4. Build a Custom Performance Analytics Platform
 - Pros:
 - Fully tailored to our specific performance testing needs
 - Maximum flexibility in data model and analysis capabilities
 - Cons:
 - Significantly higher development and maintenance effort
 - Longer time to initial value
 - Requires specialized skills to build and maintain
 - Will reinvent capabilities already available in existing tools
5. Use Object Storage (S3/GCS/Package Registry) for JSON Baselines
 - Pros:
 - Simple implementation with minimal infrastructure dependencies
 - Easy integration with CI/CD pipelines and existing tools
 - Straightforward version control of baseline files
 - Low operational overhead with highly reliable storage
 - Cost-effective for the amount of data involved
 - Cons:
 - Limited query capabilities for dynamic analysis and investigation
 - No built-in visualization or trending capabilities
 - Requires custom tooling for comparison and regression detection
 - Difficult to perform ad-hoc analysis or identify patterns across multiple tests
 - Doesn't scale well for storing full test result datasets, only suitable for baselines

We opted for the current approach because it leverages our existing infrastructure investments while enabling the dynamic baselines critical for effective MR performance testing. This solution offers the best balance of implementation speed, analytical capabilities, and long-term scalability.

References

- GitLab Performance Tool (GPT)
- GPT Benchmarks Wiki
- Reference Architecture Test Environment Details
- Replace InfluxDB with Prometheus InfluxDB exporter
 - MR
 - Issue
- Shift Left and Right Performance Testing
- End-to-End Pipeline Monitoring

SECTION: engineering/infrastructure-platforms/developer-experience/design-documents/shift_left_right_performance/_index.md

<!--

Before you start:

- Copy this file to a sub-directory and call it index.md for it to appear in the design documents list.
- Remove comment blocks for sections you've filled in.
When your document ready for review, all of these comment blocks should be removed.

To get started with a document you can use this template to inform you about what you may want to document in it at the beginning. This content will change / evolve as you move forward with the proposal. You are not constrained by the content in this template. If you have a good idea about what should be in your document, you can ignore the template, but if you don't know yet what should be in it, this template might be handy.

- Fill out this file as best you can. At minimum, you should fill in the "Summary", and "Motivation" sections. These can be brief and may be a copy of issue or epic descriptions if the initiative is already on Product's roadmap.
- Create a MR for this document. Assign it to an Architecture Evolution Coach (i.e. a Principal+ engineer).
- Merge early and iterate. Avoid getting hung up on specific details and instead aim to get the goals of the document clarified and merged quickly. The best way to do this is to just start with the high-level sections and fill out details incrementally in subsequent MRs.

Just because a document is merged does not mean it is complete or approved. Any document is a working document and subject to change at any time.

When editing documents, aim for tightly-scoped, single-topic MRs to keep discussions focused. If you disagree with what is already in a document, open a new MR with suggested changes.

If there are new details that belong in the document, edit the document. Once a feature has become "implemented", major changes should get new blueprints.

The canonical place for the latest set of instructions (and the likely source of this file) is
content/handbook/engineering/architecture/design-documents/template.md.

Document statuses you can use:

- "proposed"
- "accepted"
- "ongoing"
- "implemented"
- "postponed"
- "rejected"

-->

<!-- Design Documents often contain forward-looking statements -->

<!-- vale gitlab.FutureTense = NO -->

<!-- This renders the design document header on the detail page, so don't remove it-->

{{< engineering/design-document-header >}}

<!--

Don't add a h1 headline. It'll be added automatically from the title front matter attribute.

For long pages, consider creating a table of contents.

-->

Summary

Performance is a critical aspect of software quality that directly impacts user satisfaction and business success. We recognize that performance testing is not just a final step in development but an integral part of our entire software lifecycle. This blueprint outlines our comprehensive approach to performance testing, which goes beyond traditional load testing to encompass a wide range of techniques and practices.

We believe in shifting performance testing both left (early in development) and right (into production monitoring) to create a culture of continuous performance awareness. By integrating performance considerations at every stage - from unit testing to production monitoring - we aim to identify and address performance issues early, optimize our systems proactively, and deliver a consistently high-performing product to our users.

This guide serves as a resource, showcasing our strategies, tools, and best practices for performance engineering. It demonstrates our commitment to excellence and our innovative

approach to ensuring that GitLab remains fast, responsive, and scalable as we continue to grow and evolve.

Motivation

We have been successfully using tools like GPT for load testing against Reference Architectures. We have had experiences, such as database performance testing with large datasets, that have demonstrated the benefits of shifting performance testing left in the development process.

By adopting a broader range of performance engineering practices, we can take a more proactive stance on solving performance concerns before they surface as issues in live environments. This approach aligns with our commitment to delivering excellent user experiences and maintaining GitLab's scalability as we continue to grow.

Goals

More complete documentation on Performance Engineering practices in the handbook page
Develop a plan for adopting these practices across all Engineering teams.

Create a culture of performance awareness throughout the entire software development lifecycle.

Improve early detection and resolution of performance issues.

Enhance GitLab's overall performance and scalability.

Standardize prioritization of performance issues at GitLab, so it no longer varies team to team with the lowest possible standard being 'in response to production incident'.

Ingrain performance earlier into the development process

Non-Goals

Performance becomes something that another team is responsible for owning and driving (the individual teams should have ownership of their own performance goals)

Performance is seen as a checklist / gatekeeper item that has to be done to progress

Proposal

Our proposed Shift Left and Right Performance Testing Strategy includes:

1. Shift Left approaches:

Unit Testing: Implement performance-focused unit tests using tools like benchmark-ips for Ruby.

Early Profiling: Integrate profiling tools like ruby-prof into the development pipeline.

Continuous Performance Awareness: Use instrumented tests and observability tools throughout development.

2. Shift Right approaches:

Load Testing: Continue and enhance our use of GitLab Performance Tool (GPT) for load testing.

Production Monitoring: Implement real-time observability and user-centric performance metrics in production.

Chaos Engineering: Introduce controlled failures to test system resilience.

3. Tool and technique adoption:

- Drive adoption of modules for unit-level performance testing
- Integrate profiling tools into the development pipeline
- Implement observability tools for ongoing performance monitoring

4. Training and culture:

- Develop training programs for engineers on performance testing techniques
- Establish performance benchmarks and goals for each development stage
- Create a system for sharing performance insights across teams

Design and implementation details

<!--

This section should contain enough information that the specifics of your change are understandable. This may include API specs (though not always required) or even code snippets. If there's any ambiguity about HOW your proposal will be implemented, this is the place to discuss them.

If you are not sure how many implementation details you should include in the document, the rule of thumb here is to provide enough context for people to understand the proposal. As you move forward with the implementation, you may need to add more implementation details to the document, as those may become valuable context for important technical decisions made along the way. A document is also a register of such technical decisions. If a technical decision requires additional context before it can be made, you probably should document this context in a document. If it is a small technical decision that can be made in a merge request by an author and a maintainer, you probably do not need to document it here. The impact a technical decision will have is another helpful information - if a technical decision is very impactful, documenting it, along with associated implementation details, is advisable.

If it's helpful to include workflow diagrams or any other related images. Diagrams authored in GitLab flavored markdown are preferred. In cases where that is not feasible, images should be placed under images/ in the same directory as the index.md for the proposal.

-->

Alternative Solutions

1. Maintain current GPT-focused approach:

Pros: Familiar process, less initial effort

Cons: Misses opportunities for early performance optimization, may lead to costly late-stage fixes

2. Focus solely on production monitoring:

Pros: Real-world data, less impact on development process

Cons: Reactive rather than proactive, potential for user-impacting issues

3. Implement full shift-left without shift-right:

Pros: Early issue detection, potentially faster development

Cons: May miss real-world performance issues, less comprehensive approach

The proposed Shift Left and Right strategy provides a balanced and comprehensive approach to performance testing, addressing the limitations of these alternatives.

SECTION: engineering/infrastructure-platforms/developer-experience/developer-tooling/_index.md

Mission

- Build state-of-the-art developer tools that are efficient and reliable, empowering developers to keep their development environments up-to-date painlessly.
- Enable contributors to contribute to our tools.
- Measure what matters: improvements in developer experience, efficiency, and toil reduction, using both quantitative and qualitative metrics.

Vision

The Developer Tooling team's vision is to create tools that enable GitLab team members and the wider community to contribute to GitLab efficiently, without the friction and manual toil that often come with working on large, complex software projects like GitLab.

Areas of responsibilities

mermaid
flowchart LR

DT[Developer Tooling] --focuses on--> GDK[GitLab Development Kit]
click GDK "https://gitlab.com/gitlab-org/gitlab-development-kit"
GDK --> GDKRemoteDev[GDK remote development workspace]

DT --> CanonicalTooling[gitlab-org/gitlab tooling]

CanonicalTooling --> StaticAnalysis[Static Analysis]

DT --> AuxTooling[Auxiliary tooling
<small>Useful tools for GitLab.com.</small>]
AuxTooling --> civiz[Pipeline Visualizer]
AuxTooling --> subparticle[Subparticle]

click civiz "https://gitlab.com/gitlab-org/quality/engineering-productivity/pipeline-visualizer"

click subparticle "https://gitlab.com/gitlab-org/quality/engineering-productivity/subparticle"

Team structure

Members

{{< team-by-manager-slug "mgamea" >}}

Communication

Description	Link
-----	-----
-----	-----
GitLab Team Handle	@gl-dx/developer-tooling
Slack Channel	gdevelopertooling
Team Issue Board	Team Issue Board
Issue Tracker	gitlab-org/dx/tooling/team

SECTION: engineering/infrastructure-platforms/developer-experience/development-analytics/_index.md

Common Links

Category	Handle
-----	-----
-----	-----
GitLab Group Handle	@gl-dx/development-analytics
Slack Channel	gdevelopmentanalytics
Slack Handle	@dx-development-analytics
Team Boards	Team Issues Board, Team Epics Board, Support Requests
Issue Tracker	tracker
Team Repositories	development-analytics

Mission

Our mission is to enhance developer efficiency by delivering actionable insights, optimizing pipeline performance, and building scalable productivity tools that measurably improve the software development lifecycle.

Vision

We envision a future where GitLab's development workflows are seamless, insightful, and empowered by data. The Development Analytics team will:

- Establish GitLab as the industry benchmark for measurable developer productivity
- Improve cycle time to industry-leading standards through tooling and practices

- Create intuitive, powerful analytics dashboards that drive informed development decisions
- Deploy AI-powered systems that optimize pipeline performance and resource utilization

Team members

```
{{< team-by-manager-slug "ghosh-abhinaba" >}}
```

Core Responsibilities

mermaid

flowchart LR

DA[Development Analytics Team]

DA --> MRCT[MR Cycle Time Improvement]

DA --> Tools[Tooling Maintenance]

MRCT --> Analytics[Analytics & Observability]

MRCT --> ExecTime[Pipeline Execution Time Optimization]

MRCT --> ReviewEng[Review Engagement Enhancement]

MRCT --> PipeStab[Pipeline Stability Assurance]

Tools --> Triage[Triage Ops]

Tools --> Roulette[GitLab Roulette]

Tools --> Danger[Dangerfiles]

Tools --> EPInfra[Engineering Productivity Infrastructure]

Tools --> CNG[CLI for Cloud Native GitLab deployment]

click Triage "https://gitlab.com/gitlab-org/quality/triage-ops"

click Roulette "https://gitlab.com/gitlab-org/gitlab-roulette"

click Danger "https://gitlab.com/gitlab-org/ruby/gems/gitlab-dangerfiles"

click EPInfra "https://gitlab.com/gitlab-org/quality/engineering-productivity-infrastructure"

click Analytics "https://gitlab.com/groups/gitlab-org/-/epics/16185"

click ExecTime "https://gitlab.com/groups/gitlab-org/-/epics/15989"

click ReviewEng "https://gitlab.com/groups/gitlab-org/-/epics/16028"

click PipeStab "https://gitlab.com/groups/gitlab-org/-/epics/16186"

click MRCT "https://gitlab.com/groups/gitlab-org/-/epics/16026"

click CNG "https://gitlab.com/gitlab-org/gitlab/-/tree/master/qa/gems/gitlab-cng"

Roadmap

As part of our commitment to aligning with GitLab's company goals, our team conducts a thorough review of company-level Roadmap and Objectives and Key Results (OKRs) at the beginning of each quarter. This process ensures that our efforts are strategically focused on delivering high-impact results that contribute to the broader organizational objectives. View the Development Analytics Roadmap for FY26 for detailed insights and upcoming priorities

Dashboards

Pipeline Duration Analytics

- Pipeline Duration Analytics
- Job Execution Analytics
- Pipeline Tier Analysis
- Long-Running Test Analysis

Pipeline Stability Analytics

- Main Branch Incident Analytics
- E2E Test Analytics

Note: Access to these dashboards requires appropriate permissions. Contact team leads for access requests.

How we work

Philosophy

- We prioritize asynchronous communication and a handbook-first approach, in line with GitLab's all-remote, timezone-distributed structure.
- We emphasize the Maker's Schedule, focusing on productive, uninterrupted work.
- Most critical recurring meetings are scheduled on Tuesdays and Thursdays.
- We dedicate 3-4 hours weekly for focused learning and innovation. This protected time enables the team to explore emerging technologies, conduct proof-of-concepts, and stay current with industry trends. Meeting requests during these blocks require advance notice.
- All meeting agendas can be found in the Team Shared Drive as well as in the meeting invite.

Meetings/Events

Event	Cadence	Agenda
-----	-----	-----
-----	-----	-----
-----	-----	-----
End-of-Week progress update	Once a week (Wednesday)	Summarize status, progress, ETA, and areas needing support in the weekly update in issues and Epics. We leverage epic-issue-summaries bot for automated status checks
Team meeting	Twice a month on Tuesday 4:00 pm UTC	Agenda
Monthly Social Time	Monthly on last Thursday 4:00 pm UTC	No agenda, fun gathering. Choose one of the slots based on your timezone alignment. Read Virtual team building
Quarterly Business Report	Quarterly	Contribute to team's success, learnings, innovations and improvement opportunities for each business quarter
1:1 with Engineering Manager	Weekly	Discuss development goals (see the 1:1 guidelines)
Team member's coffee chats	Once/twice a month	Optional

meetings for team members to regularly connect

|

Yearly Roadmap Planning

- Each financial year, we create a roadmap to ensure visibility and alignment.
- We conduct an intensive month-long exercise (usually in Q4) to gather input from stakeholders.
- DRIs take the lead drafting the roadmap using the roadmap prep-work template).
- Once the roadmap is approved, during our bi-weekly team meetings, we review progress, address blockers, and gather feedback on the planned roadmap work.

Iterations

Once the yearly roadmap is defined, we structure our work using GitLab Iterations within a twice-a-month iteration model. This approach ensures consistent progress tracking, clear priorities, and iterative improvements. Here are our current iteration board and previous iterations for reference. As a team, we make sure:

1. Each issue is assigned to a Development Analytics Iteration.
2. Issues that are not worked on within the iteration automatically roll over to the next iteration.
3. In every twice-a-month team meeting, we review the iteration boards and track velocity using burndown charts.

Internal Rotation & Support Requests

Internal Rotation

We use an internal rotation for support requests and other team maintenance tasks. This frees up time for other Engineers in the team to work on planned work.

Support Requests

- If one finds a bug, needs assistance, or identifies an improvement opportunity then raise support requests using the ~"group::Development Analytics" and ~"development-analytics::support-request" labels. If the issue is urgent, escalate to the designated Slack channel - gdevelopmentanalytics.
- If a request first comes through Slack, either the requester or a group::Development Analytics member should open an issue with the correct labels to ensure proper tracking and triage.
- The team reviews the support request board and prioritizes accordingly. Generally, the team reserves ~20% of weekly time for support tasks, though this may vary based on current priorities.

Tools/Repository Maintenance

- Team does not automatically watch every new issue created in each group-owned repository-use the group labels or escalate in Slack to ensure visibility.
- We highly promote self-served Merge Requests. If one already identified a fix or improvement,

we request opening an MR for faster turnaround. The ~group::development analytics maintainers will review and merge as appropriate.

- Feature work and bug fixes follow the team's current priorities.
- Find the version management rituals for ~group::development analytics owned repositories:

Repository	Release Process
----- -----	

gitlab-roulette	Version updates are not scheduled on a set cadence. A release can be cut whenever a version-update MR is submitted.
gitlab-dangerfiles	Same as above-no regular cadence; release triggered by a version-update MR.
triage-ops	A new release is initiated after merging a new commit into master.
engineering-productivity-infrastructure	Dependency update MRs are generated by Renovate bot.

Automated Label Migration

For details on label migration, see the Handbook entry for creating label migration triage policy with GitLab Duo Workflow.

SECTION: engineering/infrastructure-platforms/developer-experience/feature-readiness/_index.md

Mission

To build and maintain a framework to ensure the readiness and consistent high quality of features being developed, maintained and delivered by GitLab.

Vision

To enable GitLab's development teams to follow the same readiness criteria for each feature developed and the ability to ensure the consistent high quality of each feature throughout each stage of the SDLC.

- Optimized and efficient feedback loops
 - Automate feature readiness policies with actionable feedback
 - The use of data to ensure critical user journeys meet quality scorecard expectations and feature readiness criteria
- Reduced cognitive load
 - Simplify operation of test tools and frameworks used for e2e testing
- Improved flow state
 - Define feature readiness policies that can be adopted to use by all development teams
 - Enable self-servicing capabilities of testing tools and test failure triage

Engineers in this team support Feature Readiness projects.

Team members

Engineering Manager: Kassandra Svoboda

S.No	
1	Richard Chong (@richard.chong)
2	Sanad Liaquat (@sliquat)
3	Sofia Vistas (@svistas)

OKRs

Every quarter, the team commits to Objectives and Key Results (OKRs). The below shows current quarter OKRs and is updated regularly as the quarter progresses.

Working with us

For individual questions please reach out to the team through our Slack channels.

Slack Channels

Channel	Purpose
#gfeaturereadiness	Channel to engage with the Feature Readiness Team

How we work

Meetings and Scheduled Calls

Our preference is to work asynchronously, within our projects issues trackers.

The team does have a set of regular synchronous calls:

- Feature Readiness Team meeting
- 1-1s between the Individual Contributors and Engineering Manager

Project Management

The majority of our project management process is described at the Platforms level and is shared between all Infrastructure Platform teams. Please read this first.

Project management links

- Team board for Developer Experience: Performance Enablement
- Team project status epic
- Team roadmap epic

Status Updates

- ASYNC weekly epic project status updates

SECTION: engineering/infrastructure-platforms/developer-experience/performance-enablement/_index.md

Mission

Enable developers to proactively build performance into the system and provide insight into feature performance health pre and post deployment.

Vision

In order to transform GitLab's performance testing from reactive to proactive while fostering a culture of performance awareness we must:

- Optimize feedback loops
 - Faster times between test setup, execution, and result analysis
 - Enable engineers to detect and prevent performance issues as early as possible
 - Enable engineers to be in the know of current Feature Performance Health pre and post-deployments
- Reduce cognitive load
 - Improve documentation for better clarity and accessibility of performance testing best practices and available tooling
 - Simplify complexity in tools and dashboards used for performance awareness
- Improve flow state
 - Define feature readiness at the performance level
 - Ensure seamless self-servicing of performing performance testing at the adequate layer of the SDLC

Common Links

S.No	Section
-----	-----
-----	-----
GitLab Team Handle	@gl-dx/performance-enablement
Team Boards	Team Board

Engineers in this team support Performance Enablement projects.

Team members

Engineering Manager: Kassandra Svoboda

S.No	
-----	-----
1	Andy Hohenner
2	Brittany Wilkerson
3	Jim Baumgardner
4	John McDonnell

| 5 | Nivetha Prabakaran |
| 6 | Vishal Patel |

Primary Projects

Name	Description
GitLab Browser Performance Tool	Tool that is a SiteSpeed wrapper which measures frontend performance in browsers, providing insights into web page performance across GitLab environments.
GitLab Component Performance Tool	Tool which leverages containerization and automated testing to provide insights on individual component performance.
GitLab Performance Tool	Tool to provide performance testing of any GitLab instance.
GitLab Verify Playbook	Experimental Tool to verify that a GitLab instance is up and functional after deployment or reconfiguration.

All Projects

Name	Description
Test Data Generator	Tool designed to populate a GitLab instance with dummy data that can be used to simulate a larger production instance.
Performance Test Data	This Project serves as an LFS data repository for the GitLab Performance Tool
Performance Docker Images	Docker builder and registry for GitLab Performance testing
AI Gateway Latency Baseline Executor	Gets the latency baseline for AI Gateway in a specific region

Working with us

To request for help with performance testing of a new feature, please create a new issue within the GPT project with the request for help template.

For individual questions please reach out to the team through our Slack channels.

Slack Channels

Channel	Purpose
gperformanceenablement	Channel to engage with the Performance Enablement Team

How we work

Meetings and Scheduled Calls

Our preference is to work asynchronously, within our projects issues trackers.

The team does have a set of regular synchronous calls:

- Performance Enablement Team meeting

- 1-1s between the Individual Contributors and Engineering Manager

Project Management

The majority of our project management process is described at the Platforms level and is shared between all Infrastructure Platform teams.

Project management links

- Team Project Status epic
- Team Roadmap epic

SECTION: engineering/infrastructure-platforms/developer-experience/test-governance/_index.md

Common Links

Category	Handle
----- -----	

GitLab Group Handle	@gl-dx/test-governance
Slack Channel	gtest-governance
Slack Handle	@dx-development-analytics
Team Boards	
Issue Tracker	tracker
GitLab Repositories	test-governance

Mission

Vision

Team members

{{< team-by-manager-slug "amyphillips" >}}

Core Responsibilities

mermaid

graph LR

A[Test Governance Team]

A --> B[Provide test expertise for critical Product releases]

B --> B1[Define test strategy for the critical feature releases]

B --> B2[Ensure adequate and robust test coverage for critical features]

B --> B3[Stay informed about the stage roadmap]

A --> C[Stage-level test tools and infrastructure]

C --> C1[Design, Build and continuously improve tests, test frameworks and tools]

C --> C2[Influence, advise and increase the testing capabilities for product teams]

C --> C3[Monitor and address test flakiness]

A --> D[Test guidance. Provide guidance and coach engineering teams in the areas:]

D --> D1[Writing e2e tests and feature specs]

D --> D2[Debugging and fixing test failures]

D --> D3[Planning testing early in the development process]

D --> D4[Anticipating test infrastructure needs and requesting change in advance]

D --> D5[Shift left and maintaining appropriate ratio between unit, integration and e2e tests]

D --> D7[Post-incident action]

A --> E[E2E Pipeline triage. Shared responsibility among all DevX sub-department]

Roadmap

How we work

Work related rituals

Work management

Planning

Working with us through request for help

The Test Governance group aims to better enable teams to apply the principle that quality is everyone's responsibility.

Please request all support via the RFH process below. This will allow us to prioritize requests against our planned project roadmap.

Please use the following Request for Help process for all support requests.

Request for Help Process

1. Creates an issue in the Request for Help project. Please complete all sections of the template so we can quickly triage your request

1. The Test Governance team will triage the request within a week, adding appropriate labels and assigning team members based on the request type and priority. You will receive details about the prioritization and next steps on the request for help issue.

For more detailed guidance on E2E test coverage, consider these approaches:

Engage with key DRIs to define persona use cases that illustrate how different customers will use new features

Evaluate which parts of use cases can be covered by lower-level tests versus E2E tests, keeping the entire testing pyramid in mind

Refer to our documentation on Testing Best Practices before submitting your request

SECTION: engineering/infrastructure-platforms/gitlab-delivery/_index.md

This is a draft page and it is currently a work in progress

Mission

The GitLab Delivery mission is to streamline and optimize the delivery, release, and deployment of GitLab across all platforms: SaaS, Self-Managed, and Dedicated. We aim to enhance the reliability, scalability, and efficiency of our processes, enabling customers to seamlessly adopt, deploy, and scale GitLab solutions. By evolving our tools, automation, and architectural practices, we are committed to delivering a consistent, secure, and high-quality experience that empowers both internal teams and customers to achieve more, faster.

The GitLab Delivery Stage Vision is defined in the Infrastructure Platforms direction pages

Vision and Direction

Our vision and direction are located with other product direction pages here.

Roadmap

Stage roadmap will be listed here

<!-- Link to Roadmap -->

Top-level Responsibilities

GitLab Delivery stage is responsible for ensuring seamless, reliable, and efficient delivery of high-quality software. We focus on optimizing workflows, enabling collaboration, and driving continuous improvement to meet customer needs

Teams

The GitLab Delivery Stage is composed by the following teams:

- GitLab Build
- Self-Managed (subject to change)
- Framework
- GitLab Release
- GitLab Deploy

<!-- to add links to single teams pages -->

Performance indicators

TBD

On-call duties

- Release Manager ownership
- Build and Deploy Pipeline DRI

Project Management

GitLab Delivery Stage has an entry level Epic that collects all the groups roadmaps and projects in progress as a single view. This epic is also used for the weekly Grand Review

Issue Boards

Each working epic should be accompanied by an issue board. Issue boards should be tailored to the specific project needs, but at minimum it should contain the workflow labels shown on the workflow diagram.

<!-- Labels -->

<!-- Labels explanations -->

<!-- Workflow -->

<!-- Priority Labels -->

<!-- Priority labels explanations -->

<!-- Other Labels -->

GitLab Delivery AMA and Demo

Needs to be scheduled, starting in January 2026

History

Stage history will come here

<!-- Add here how the Stage came into existence -->

SECTION: engineering/infrastructure-platforms/gitlab-delivery/build/_index.md

Overview

The Build team focuses on artifacts our customers use to deploy GitLab. This includes system packages, container images, and packages on cloud provider marketplaces. Build team members serve as the bridge between development teams and the end users. Build engineers provide tools and expertise that enable efficient component builds and deliver a simple, secure, and dependable installation experience.

The Build team also reviews a large number of externally authored merge requests. Bundled components such as PostgreSQL, Consul, and Patroni require regular dependency and security updates. Community members may also request changes to compile flags that enable specific

optimizations.

Our Customers

Internal Customers - Development Teams

We partner with GitLab's development teams to:

- Integrate their components into cohesive installation packages
- Provide reliable build tools and automation
- Ensure secure dependency management and vulnerability mitigation
- Streamline the path from code to packaged software

External Customers - GitLab Users

We deliver exceptional installation experiences with:

- Reliable and well-tested installation methods across supported platforms
- Secure and up-to-date packages, containers, and cloud provider marketplace images
- Clear documentation and upgrade paths
- Compatibility across diverse customer environments

Mission

The Build Team ensures GitLab components are seamlessly integrated, securely packaged, and reliably delivered to our customers. We serve as the bridge between development teams and end users, providing tooling and expertise that enables efficient component building while delivering installation experiences that are simple, secure, and dependable.

Vision

Our vision and direction aligns with the direction of the broader Delivery Group.

For Internal Customers (Development Teams)

- Every development team has access to reliable, efficient build tools that accelerate their workflow
- Component integration is automated, secure, and requires minimal manual intervention
- Security vulnerabilities are proactively identified and mitigated across all dependencies
- Build processes are consistent, well-documented, and self-service where possible

For External Customers (GitLab Users)

- GitLab installs effortlessly across all supported platforms and architectures
- Ship production-ready and thoroughly tested packages.
- Customer deployments upgrade reliably with minimal intervention
- Security patches release rapidly and transparently

Technology Vision

- GitLab has an official installation method on all supported platforms and architectures
- GitLab offers official one click installation method on all supported cloud platforms
- GitLab is able to automatically upgrade itself safely and reliably
- GitLab runs equally well on both high and low resource systems (such as Raspberry Pi)
- GitLab.com is running using the official installation methods
- Any GitLab installation is able to report installation/upgrade errors automatically
- Setting up GitLab in HA configuration is automated and simple
- All installation methods are automatically tested before release

Team Vision

- Each team member can work on all team projects in addition to their domain expertise
- Each team member strives to identify candidates that strengthen the team and raise the quality of our work.
- Team creates and improves documentation to facilitate knowledge transfer and enable self-service
- Based on GitLab's leadership principles, our team is able to reach a conclusion independently all the time, consensus most of the time
- Team members are supported in pursuing certificati